CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS · DOMAIN 1

Building Agentic Applications with the Claude Agent SDK

A step-by-step technical walkthrough: the agent loop · tool & MCP integration · multi-agent orchestration · subagent delegation · lifecycle hooks · production controls

AAA Learning Group · engineering deep-dive · sources cited inline (↗) throughout

What we'll cover — and how to read this deck

Framing

Workflows vs agents; when NOT to build one

01

The agent loop

stop_reason, turns, message types, controls

02

Tools & MCP

custom tools, ACI design, scoping

03

Orchestration

coordinator + subagent delegation

04

Lifecycle hooks

deterministic control & normalization

05

Production

budget, compaction, sessions, observability

06

These six blocks map directly to Domain 1 (Tasks 1.1–1.7) plus the Domain 2 / Domain 5 ideas the exam threads through agentic scenarios.

↗ *anthropic.com/engineering/building-effective-agents*

First principle: workflows vs agents

Anthropic's guidance: start with the simplest thing that works; add agency only when it demonstrably improves outcomes. Agents trade latency and cost for flexibility.

Workflow

LLM + tools orchestrated through predefined code paths.

- Predictable, consistent, debuggable
- Best for well-defined tasks
- You own the control flow

Agent

LLM dynamically directs its own process & tool use in a loop.

- Flexible, model-driven, scales to open-ended work
- Best when steps can't be predicted
- Higher cost; risk of compounding errors

Senior takeaway: “agentic” is a spectrum. Reach for full autonomy only for open-ended tasks in trusted environments with good feedback signals. Everything in this deck applies to both — the SDK runs the loop either way.

↗ *anthropic.com/engineering/building-effective-agents*

The composable patterns you'll combine

Prompt chaining

Fixed sequential steps; gate between them.

Use for: *Predictable multi-aspect work*

Routing

Classify input → specialized handler.

Use for: *Distinct categories, separate prompts*

Parallelization

Sectioning or voting, aggregated.

Use for: *Speed or multiple perspectives*

Orchestrator–workers

Coordinator splits & synthesizes dynamically.

Use for: *Subtasks can't be predicted*

Evaluator–optimizer

Generate → critique → refine in a loop.

Use for: *Clear criteria, iterative value*

Where Domain 1 lives

The exam's multi-agent research system is orchestrator–workers; the support agent is a routed agent with enforcement. You'll build both today.

↗ platform.claude.com/docs/en/agent-sdk/overview

What the Claude Agent SDK gives you

The same runtime that powers Claude Code, as a library. It runs the loop and hands you control over tools, permissions, cost limits, sessions, subagents, and hooks. No CLI required.

Runs the loop

Evaluate → call tools → feed results back → repeat.

Tools & MCP

Built-in tools + custom tools via in-process MCP.

Subagents

Delegate to isolated, specialized agents.

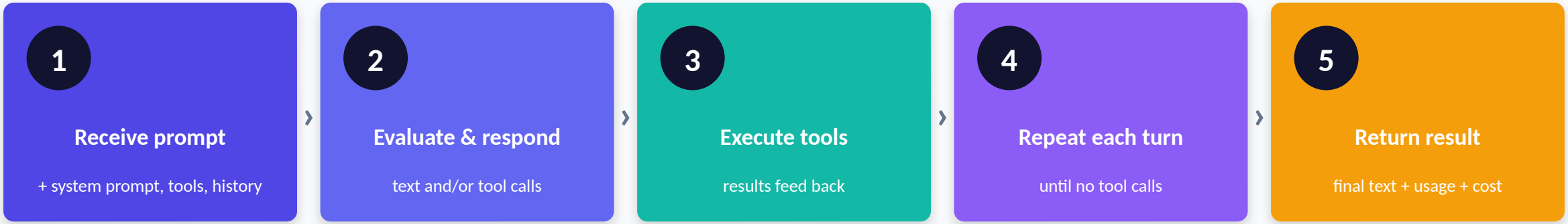
Control plane

Permissions, budget, effort, hooks, sessions.

Python `pip install claude-agent-sdk` **TypeScript** `@anthropic-ai/claude-agent-sdk` *(renamed from "Claude Code SDK", Sept 2025)*

↗ *platform.claude.com/docs/en/agent-sdk/agent-loop*

The agent loop, step by step



One turn = Claude produces output with tool calls → SDK executes them → results return automatically, without yielding to your code. Steps 2–3 loop until Claude responds with **no tool calls**.

Example — "Fix the failing tests in auth.ts": Turn 1 Bash npm test → Turn 2 Read files → Turn 3 Edit + re-run (green) → final text-only turn. Four turns, three with tools.

↗ platform.claude.com/docs/en/agent-sdk/agent-loop#message-types

Reading the loop: message types

As the loop runs the SDK yields a stream of typed messages. Knowing them is how you build progress UIs, audit trails, and cost tracking.

`SystemMessage`	Session lifecycle — subtype init / compact_boundary / …
`AssistantMessage`	After each Claude response — text + tool-call blocks
`UserMessage`	After each tool execution — the tool result sent back
`StreamEvent`	Raw streaming deltas (only when partial messages on)
`ResultMessage`	End of loop — final text, token usage, cost, session_id

Drive most apps off ResultMessage (output + cost + subtype). Stream AssistantMessage for live progress.

↗ *docs: build-with-claude/handling-stop-reasons*

Walkthrough 1 — the loop on stop_reason

```
messages = [{"role":"user","content": prompt}]
for _ in range(max_safety_turns):     # circuit breaker
    r = client.messages.create(model, tools=TOOLS,
                               messages=messages)
    messages.append({"role":"assistant",
                     "content": r.content})
    if r.stop_reason == "end_turn":   # 1 done
        return final_text(r)
    if r.stop_reason == "tool_use":   # 2 continue
        results = run_tools(r.content)
        messages.append({"role":"user",
                 "content": results})  # 3 feed back
```

1 Terminate on end_turn

Return the model's final text — not on assistant text or an iteration cap.

2 Continue on tool_use

Run the requested tools, then loop again with the results.

3 Feed results back

Append tool_result blocks (keyed by tool_use_id) as a user turn.

Loop control: correct vs anti-patterns

Correct practice

- Branch on stop_reason every turn
- Append tool results to history so the model reasons over them
- Let Claude choose the next tool from context (model-driven)
- Keep a high turn/budget cap purely as a circuit breaker

Anti-patterns

- Parse natural-language text to decide it's “finished”
- Use an iteration cap as the PRIMARY stop mechanism
- Treat any assistant text as a completion signal
- Hard-code a fixed tool sequence / decision tree

↗ platform.claude.com/docs/en/agent-sdk/agent-loop#control-how-the-loop-runs

Controlling the loop in production

`max_turns`

Cap tool-use round trips. Prevents runaway sessions on open-ended prompts.

`max_budget_usd`

Stop at a spend threshold. The recommended default for production agents.

`effort`

low → max reasoning per turn. Use low for file lookups; high for refactors/debugging.

`permission_mode`

default / acceptEdits / plan / dontAsk / bypassPermissions — gate what runs without asking.

Hit a limit and the loop ends with a ResultMessage subtype (error_max_turns / error_max_budget_usd) — capture session_id and resume with a higher cap.

↗ platform.claude.com/docs/en/agent-sdk/agent-loop#handle-the-result

Handling the result — every termination state

`success`	Task finished normally — result text present
`error_max_turns`	Hit maxTurns — resume with a higher cap
`error_max_budget_usd`	Hit budget — decide whether to extend
`error_during_execution`	API failure / cancellation interrupted the loop
`error_max_structured_output_retries`	No valid structured output within retry limit

Always check subtype before reading result. *All subtypes carry total_cost_usd, usage, num_turns, session_id — so you can track spend and resume even after a failure.*

↗ *platform.claude.com/docs/en/agent-sdk/custom-tools*

Tool integration — step by step

1

`@tool`

Name, description, input schema, async handler.

2

`create_sdk_mcp_server`

Bundle tools into one in-process MCP server.

3

`allowed_tools`

Pre-approve mcp__<server>__<tool> names.

4

`is_error`

Return failures as data the loop can recover from.

Tools run in-process inside your app — your database client, your API auth, your domain logic. Scope each agent to ~4–5 tools; selection reliability degrades as the set grows.

↗ *platform.claude.com/docs/en/agent-sdk/custom-tools*

Walkthrough 2 — a custom tool

```python
@tool(
  "lookup_order",            # 1 namespaced name
  "Look up an ORDER by order ID (O-###)."
  " Returns total, status, customer. NOT"
  " for customer lookups.",   # 2 selection signal
  {"order_id": str})          # typed input schema
async def lookup_order(args):  # handler
    rec = _ORDERS.get(args["order_id"])
    if not rec:
        return {"content":[...],
                "is_error": True}  # 3 recoverable
    return {"content":[{"type":"text",...}]}
```

1 Namespaced name

Becomes mcp__support__lookup_order; list it in allowed_tools and scope per agent.

2 Description = selection signal

Distinct, bounded text — the model routes between tools on this.

3 Structured errors

is_error keeps the loop alive so the agent can recover.

Agent–computer interface (ACI) design

Invest in your tool interface as much as your prompt. “Put yourself in the model's shoes” — a tool should be as obvious to Claude as a great docstring is to a new teammate.

Rich descriptions

Input formats, example queries, edge cases, and explicit boundaries vs similar tools — descriptions are the primary selection mechanism.

Split, don't overload

Break a generic analyze_document into extract_data_points, summarize_content, verify_claim. Rename to remove overlap.

Scope per agent

~4–5 tools per role. Tools outside an agent's specialization get misused (a synthesis agent that web-searches).

Poka-yoke the args

Make mistakes impossible: require absolute paths, validate URLs, constrain enums. Anthropic spent more time on tools than prompts for SWE-bench.

↗ platform.claude.com/docs/en/agent-sdk/agent-loop#tool-execution

Built-in tools, MCP, and context cost

File & search

`Read · Write · Edit · Glob · Grep`

Execution & web

`Bash · WebSearch · WebFetch`

Orchestration

`Agent · Skill · AskUserQuestion · TaskCreate/Update`

External / custom

`MCP servers + your @tool handlers`

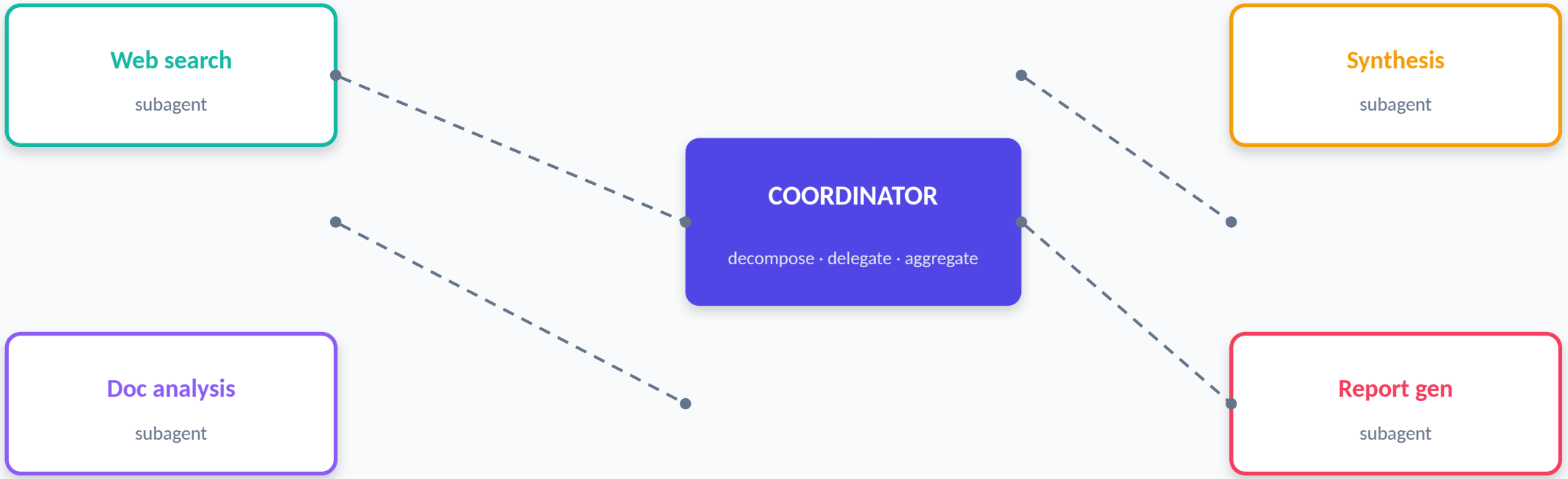
Context is a budget you spend before the agent does any work

- Every tool definition consumes context on every request — another reason to scope tightly.
- MCP tool search defers MCP schemas and loads them on demand; without it, each server adds all its schemas up front.
- Read-only tools (Read/Glob/Grep, MCP tools marked read-only) can run in parallel; state-changing tools run sequentially.
- Mark custom tools readOnlyHint to allow parallel execution.

↗ platform.claude.com/docs/en/agent-sdk/subagents

Multi-agent orchestration: hub & spoke

A coordinator owns ALL inter-agent communication, routing, and error handling. Subagents never talk to each other — everything flows through the hub for observability and control.



What a good coordinator does

Dynamic selection

Analyze the query and invoke only the subagents it needs — don't always run the full pipeline.

Broad decomposition

Partition the WHOLE topic into non-overlapping subtasks. Narrow decomposition silently drops coverage.

Iterative refinement

Evaluate synthesis for gaps, re-delegate targeted queries, re-synthesize until coverage is sufficient.

Route everything

All subagent comms go through the hub for observability and consistent error handling.

↗ *platform.claude.com/docs/en/agent-sdk/subagents*

Walkthrough 3 — subagent delegation

```
options = ClaudeAgentOptions(
  allowed_tools=["WebSearch","Task","Agent"],  # 1
  agents={
    "web-researcher": AgentDefinition(   # 2
      description="Searches ONE subtopic; cited.",
      prompt="Return claim+excerpt+URL+date...",
      tools=["WebSearch","Read"],   # 3 scoped
      model="sonnet", effort="medium"),
    "synthesizer": AgentDefinition(
      description="Combines findings into report.",
      tools=["Read"]),   # scoped: no web
  })
```

1 **Include the spawn tool**

Coordinator allowed_tools MUST list Task/Agent.

2 **AgentDefinition**

description, prompt, tools, model, effort, skills, mcpServers.

3 **Scope per role**

Researcher searches; synthesizer can only Read.

↗ subagents#what-subagents-inherit

Subagents run with isolated context

A subagent starts a FRESH conversation. The only channel from parent to child is the spawn prompt string — pass every file path, finding, and decision it needs directly in that prompt. Only the subagent's final message returns to the parent.

Subagent RECEIVES

- Its own system prompt (AgentDefinition.prompt)
- The spawn prompt string from the parent
- Tool definitions (all, or the scoped subset)
- Project CLAUDE.md (if setting sources enabled)

Subagent does NOT receive

- The parent's conversation history or tool results
- The parent's system prompt
- Skills (unless listed in the definition)
- Any shared live memory between invocations

Context isolation is the FEATURE: a subagent can read 30 files and only a concise summary returns — the parent's context stays lean. Subagents also cannot spawn their own subagents.

↗ *subagents#detecting-subagent-invocation*

Spawning: the Task→Agent rename + parallelism

Know both names

The exam guide calls the spawn tool **`Task`** and requires allowedTools to include `"Task"`. It was renamed to **`Agent`** in Claude Code v2.1.63; current SDKs emit "Agent" in tool_use blocks but still list "Task" in system:init. **Include and match BOTH.**

Spawn in parallel — emit multiple spawn calls in ONE coordinator response

Parallel ✓

One turn → 3 spawn calls → run concurrently. Big latency win for independent searches.

```
[Task(a), Task(b), Task(c)]  # single response
```

Sequential ✗

One spawn per turn forces round-trips and slows the whole pipeline unnecessarily.

```
turn1: Task(a); turn2: Task(b); ...
```

Passing context & preserving provenance

Pass complete findings

Include prior agents' outputs directly in the next subagent's prompt (search + analysis → synthesis).

Separate content from metadata

Carry source URL, doc name, page, and date alongside each claim so attribution survives synthesis.

Goals over procedures

Coordinator prompts state research goals & quality criteria, not rigid scripts — so subagents adapt.

Structured handoffs

On escalation, compile a summary (IDs, root cause, recommended action) for humans without the transcript.

Failure mode: narrow decomposition

Scenario: Topic “impact of AI on creative industries.” Every subagent succeeds, yet reports cover only visual arts. Logs show the coordinator split it into *digital art*, *graphic design*, *photography* — music, writing, and film were never assigned.

Root cause: the coordinator's task decomposition was too narrow. The subagents executed correctly within the scope they were given.

Why not the distractors

- Synthesis agent — it faithfully synthesized what it was given
- Web-search agent — it found relevant articles for its assigned slices
- Doc-analysis agent — it summarized correctly within scope

Diagnostic heuristic: when every worker succeeds but coverage is wrong, suspect the decomposition — not the workers.

TASK 1.5

↗ platform.claude.com/docs/en/agent-sdk/hooks

Lifecycle hooks — control points

Hooks are callbacks that fire at agent events. They run in YOUR process (not the model's context) and give deterministic control where prompts can only ask nicely.

EVENT	FIRES	USE FOR
`PreToolUse`	Before a tool runs	Allow / deny / modify input — block refunds > $500
`PostToolUse`	After a tool returns	Normalize data, append context, replace output
`SubagentStart / Stop`	Subagent lifecycle	Track parallel spawning; aggregate results
`UserPromptSubmit`	On user prompt	Inject extra context into the prompt
`Stop / PreCompact`	End / before compaction	Save state; archive transcript before summarizing

↗ platform.claude.com/docs/en/agent-sdk/hooks

Walkthrough 4 — PreToolUse enforcement

```python
async def enforce_refund_limit(input, id, ctx):
  if input["tool_name"]=="mcp__ops__process_refund":
    amt = float(input["tool_input"]["amount"])
    if amt > 500:                          # 1 the rule
      return {"hookSpecificOutput": {
        "hookEventName": input["hook_event_name"],
        "permissionDecision": "deny",     # 2 blocks
        "permissionDecisionReason":         # 3 reason
          "Refunds > $500 need a human."}}
  return {}   # allow everything else
```

1 Match the rule

Inspect the args before the tool runs — here, amount > $500.

2 deny blocks the call

permissionDecision: allow | deny | ask | defer.

3 Give a reason

The model sees it and won't blindly retry. deny always wins if hooks conflict.

PreToolUse can also rewrite a call: return updatedInput (with permissionDecision "allow") to sandbox a path or sanitize arguments before execution.

TASK 1.5

↗ platform.claude.com/docs/en/agent-sdk/hooks#outputs

PostToolUse — normalize heterogeneous data

Different MCP tools return dates as Unix timestamps, ISO-8601, or numeric codes. A PostToolUse hook normalizes results BEFORE the model reasons over them — removing a whole class of confusion.

```python
async def normalize_timestamps(input, id, ctx):
  if input["tool_name"]=="mcp__ops__lookup_order":
    data = json.loads(raw_text(input))
    iso = to_iso8601(data["ordered_at"])  # 1719500000→
    return {"hookSpecificOutput": {
      "hookEventName": input["hook_event_name"],
      "additionalContext":                  # append
        f"Normalized ordered_at (ISO): {iso}"}}
  return {}
```

Fires after execution

Transform results the model hasn't seen yet.

additionalContext

Append normalized info to the tool result (matches the highlighted line).

updatedToolOutput

Replace the raw output entirely when needed.

Hooks vs prompts: deterministic vs probabilistic

Hooks / prerequisite gates

- Deterministic — the rule ALWAYS holds
- Code-enforced, outside the model
- Right for must-hold business rules
- e.g. block refunds > $500; verify identity first

Prompts / few-shot

- Probabilistic — non-zero failure rate
- Great for judgment, tone, and guidance
- NOT sufficient when errors cost money
- Use to shape behavior, not to guarantee it

Exam rule of thumb: if a business rule MUST hold (identity before refund, dollar limits, ordering of steps), enforce it with a hook or prerequisite gate — never with a stronger prompt or more examples. This single distinction resolves a large share of Domain 1 scenario questions.

Walkthrough 5 — programmatic prerequisite gate

Problem: in 12% of cases the agent skips get_customer and looks up orders by stated name → wrong account, wrong refund. The fix is a programmatic prerequisite, not a stronger prompt.

```
VERIFIED = set()    # state OUTSIDE the model
GATED = {"mcp__support__lookup_order",
         "mcp__support__process_refund"}

async def require_verification(input, id, ctx):
  if input["tool_name"] in GATED and not VERIFIED:
    return deny("Call get_customer to verify first")
  return {}
```

A ✓ Prerequisite gate

Deterministic ordering — correct.

B ✗ "Mandatory" prompt

Probabilistic; still fails sometimes.

C ✗ Few-shot examples

Probabilistic; doesn't guarantee.

D ✗ Routing classifier

Changes availability, not ordering.

↗ platform.claude.com/docs/en/agent-sdk/agent-loop#the-context-window

Context window & automatic compaction

Context does not reset between turns — system prompt, tool defs, history, and tool outputs all accumulate. Large tool outputs are the usual culprit.

Auto-compaction

Near the limit, the SDK summarizes older history (emits a compact_boundary message). Early instructions may be lost.

Persistent rules → CLAUDE.md

Re-injected every request, so they survive compaction — unlike instructions buried in the first prompt.

Subagents for heavy work

Isolate verbose exploration; only the summary returns to the parent's context.

Trim & tune

Keep only relevant tool fields; lower effort for routine turns; PreCompact hook to archive transcripts.

↗ *platform.claude.com/docs/en/agent-sdk/sessions*

Sessions: continue, resume, fork

Continue

Pick up the MOST RECENT session in the directory. No ID tracking. One conversation at a time.

Resume

Return to a SPECIFIC session by ID. Needed for multi-user apps or recovering after a limit/restart.

Fork

Branch a COPY of history to try a different approach; the original stays untouched. Two independent IDs.

Production notes

- Capture session_id from ResultMessage (present even on errors). Python's ClaudeSDKClient tracks it for you.
- Sessions persist the CONVERSATION, not the filesystem — use file checkpointing to revert file changes.
- Resuming across hosts: move the .jsonl (cwd must match), or — often more robust — inject a structured summary into a fresh session.

Error propagation across agents

When a subagent fails (e.g. web search times out), return STRUCTURED error context so the coordinator can recover intelligently — retry, try an alternative, or proceed with partial results.

Return structured context ✓

- failure type (timeout, validation, permission)
- the attempted query
- any partial results gathered
- possible alternative approaches
- isRetryable flag for the coordinator

Anti-patterns ✗

- Generic “search unavailable” — hides context
- Empty results marked as success — silent failure
- Kill the entire workflow on one failure
- Confuse access failures with valid empty results

Operating agents in production

Observe

Stream message types; SubagentStart/Stop + PostToolUse hooks for audit trails. Hooks don't cost model context.

Bound cost

max_budget_usd + per-turn effort; read total_cost_usd / usage off every ResultMessage.

Constrain

permission_mode + allow/deny rules + PreToolUse gates. bypassPermissions only in isolated envs.

Recover

Capture session_id; resume on limits; structured errors; checkpoints for file rollback.

Reference architectures

Customer support agent

Single routed agent · MCP tools · 80%+ first-contact resolution

- Tools: get_customer, lookup_order, process_refund, escalate_to_human
- Prerequisite gate: verify identity before order/refund ops
- PreToolUse hook: cap refunds; redirect to escalation
- Escalate on explicit request, policy gaps, no progress
- Persist a 'case facts' block; trim verbose tool output

Multi-agent research

Orchestrator–workers · search / analysis / synthesis / report

- Coordinator decomposes broadly; spawns searchers in parallel
- Subagents: isolated context, scoped tools, cited findings
- Pass findings + metadata explicitly to synthesis
- Structured error context → recover with partial results
- Annotate coverage gaps and conflicting sources

Exam cheat sheet — traps & truths

Loop control

Drive on stop_reason — not text, not iteration caps.

Tool results

Append as a user turn with tool_result + tool_use_id.

Spawning

Coordinator needs Task/Agent; subagents can't recurse.

Subagent context

Isolated — pass everything in the spawn prompt.

Parallelism

Multiple spawn calls in ONE response.

Hooks vs prompts

Must-hold rule → hook; judgment → prompt/few-shot.

Tool count

Scope tightly; ~4–5 beats 18 for reliability.

Errors

Structured context > generic 'failed'; is_error to recover.

Production

Set max_budget_usd; check ResultMessage subtype.

Continuity

Stale results → fresh session + summary, not resume.

PRACTICE

↗ *Topic1 folder · /code*

Hands-on lab — code walkthrough files

`01_agentic_loop_raw_api.py`

Bare-metal loop on stop_reason.

`02_custom_tools_sdk.py`

@tool + in-process MCP server.

`03_coordinator_subagents.py`

Coordinator + scoped subagents.

`04_lifecycle_hooks.py`

PreToolUse block + PostToolUse normalize.

`05_prerequisite_gate.py`

Deterministic verify-before-refund gate.

`Agent_SDK_Walkthrough.ipynb`

Colab notebook — run all five live.

Try it: `pip install anthropic claude-agent-sdk` · set ANTHROPIC_API_KEY · then run each file or open the notebook in Colab. Pair with the 15-question MCQ set.

KEY TAKEAWAYS

Six things to carry into the exam — and into prod

- Choose the simplest architecture; agency is a cost/latency trade, not a default.
- The agent IS the loop — drive on stop_reason; the SDK runs it, you control it.
- Orchestrate hub-and-spoke; coordinators decompose broadly and route everything.
- Subagents = isolated context + scoped tools; pass what they need explicitly.
- Hooks give deterministic guarantees; use them whenever a rule must always hold.
- Operate it: budget, permissions, observability, sessions, structured errors.

Go deeper: platform.claude.com/docs/en/agent-sdk (overview · agent-loop · subagents · hooks · custom-tools · sessions) · anthropic.com/engineering (building effective agents · multi-agent research). Full annotated list in Reference_Links.md.

Questions & discussion

Next: run the Colab lab → take the 15-question MCQ set → bring edge cases back to the group.